

FOI2021 算法高级在线 1-2 详解

周成 陈亮舟

(福建师范大学附属中学 福州 350007)

摘 要 题目来源于 FOI 模拟赛。本文对三个问题的解题思路及算法设计作了较为详细的描述, 并给出相应的代码。

关键词 题解; 分析; 算法; 代码

中图法分类号 TP31 DOI:10.16707/j.cnki.fjpc.2021.07.051

The Solutions of the Day 1-2 Tasks for FOI2021

ZHOU Cheng, CHEN Liangzhou

(the Affiliated High School of Fujian Normal University, Fuzhou, China, 350007)

1 数排列

表 1 数据规模与约定

测试点编号	$n \leq$	特殊性质
1	8	无特殊性质
2		
3	500	
4		
5		
6		
7	2.5×10^5	$x_i \leq x_{i+1}$
8		
9		
10		
11		$x_2 = 1$
12		
13		
14		
15		无特殊性质
16		
17		
18		
19		
20		

1.1 问题描述

给一个长度为 m 的序列 x , 值域在 $[1, n]$ 之间且互不相同。

求字典序最小的长度为 m 的子序列为 x 的 $[1, n]$ 排列的数量。

答案对 998244353 取模。

1.1.1 涉及的知识点

性质挖掘, 组合计数。

1.1.2 数据规模与约定

数据规模与约定如表 1 所示。对于所有的数据, $1 \leq m \leq n \leq 2.5 \times 10^5$, $1 \leq x_i \leq n$, 且 x_i 互不相同。

1.2 解题思路

考察长度为 n 的排列 a 的字典序最小的长度为 m 的子序列 x 的性质。

要满足字典序最小的条件仅需进行贪心的过程: 对于第 1 位 (x_1):

(1) 能否取到 1 (可能的最小数) 作为第 1 位?

(2) 需要满足什么条件才可能?

①如果可能, 对周围的数有什么限制?

②如果不可能, 那可能取到什么数?

注意到, 子序列的第 1 位必须取在下标区间

$[1, n - m + 1]$ 中。为了让字典序最小,显然应该取在这个区间内的最小数。例如:若数值1恰好在这个区间内,则取1即可。

取完第1位后,设 x_1 取在第 p 位,右边 $[p + 1, n]$ 就是一个子问题。此时发现第2位取在下标区间 $[p + 1, n - m]$ 中的最小数。和第1位的 $[1, n - m + 1]$ 相比,右端点增加了1位。

因此若 x_2 取在除了那一位下标的情况,一定有 $x_2 > x_1$ 。也就是说,如果 $x_2 < x_1$,则一定是 x_2 取在了下标 $n - m$ 上。

考虑 $x_2 > x_1$ 的情况,此时可以继续考虑 $x_3, x_4, \dots$ 。和之前类似,直到找到了一个 $x_i < x_{i-1}$ 的情况。如果出现了 $x_i < x_{i-1}$,则一定是 x_i 取到了排列的第 $n - m + i$ 位。

可以发现,后续的 $x_{i+1} \sim x_m$ 已经没有选择的余地,即它们必须形成一个原排列的后缀。

此时,已经对给定排列 a ,构造最小字典序子序列 x 的算法有一定理解。

接下来考虑给定 x 时 a 的结构。

先考虑 x 递增的情况。

假设 $x_1 \sim x_m$ 在 a 中依次位于 $p_1 \sim p_m$ 的下标位置上。

由从 a 构造 x 的过程,可以发现:

- (1) 区间 $[1, p_1)$ 中的数必须 $> x_1$ 。
- (2) 区间 $(p_1 + 1, p_2)$ 中的数必须 $> x_2$ 。
- (3)
- (4) 区间 (p_{m-1}, p_m) 中的数必须 $> x_m$ 。
- (5) 区间 $(p + m, n]$ 中的数也必须 $> x_m$ 。

可以确认这些条件的充分性和必要性。此时,不能枚举 $p_1 \sim p_m$,这样时间复杂度太高。

一个合适的方法是,先令序列 $a = x$,然后从小到大插入不在 x 中的数,最后形成一个排列。

由于先插入的数能插入的位置完全包含在后插入的数能插入的位置中,可直接导出一个性质:每个数能插入的位置数量不取决于之前的插入方式,比它先插入了多少个数,它能插入的位置就增加多少(因为数之间的间隔变多了)。这时很容易在 $O(n)$ 时间内解决这个问题。

对于 x 不一定递增的情况,找到第一个满足 $x_i < x_{i-1}$ 的位置。由于之前的性质, $x_i \sim x_m$ 都必须是 a 的后缀。此时考虑前缀 $[1, n - (m - i + 1)]$, $x_1 \sim x_{i-1}$ 都必须落在其中。可以发现,此时和 x 递增的情况相同。

时间复杂度 $O(n)$ 。

1.3 总结

作为省队选拔模拟赛第一题,本题做法相对简单,但也考察了选手对问题的转化能力和发现问题性质的能力,以及在组合计数方面的一类处理方式的掌握程度^[1]。

1.4 参考代码

```
#include <cstdio>
```

```
typedef long long LL;
```

```
const int Mod = 998244353;
```

```
const int MN = 250005;
```

```
int N, M, b[MN];
```

```
int main() {
```

```
    scanf("%d%d", &N, &M);
```

```
    int lst = 0, mxv = 0;
```

```
    for (int i = 1; i <= M; ++i) {
```

```
        int x;
```

```
        scanf("%d", &x);
```

```
        if (x > lst) {
```

```
            b[x] = i;
```

```
            lst = x;
```

```
            mxv = x;
```

```
        } else {
```

```
            b[x] = -1;
```

```
            lst = N;
```

```
        }
```

```
    }
```

```
    int Ans = 1, V = 0;
```

```
    for (int i = 1; i <= N; ++i) {
```

```
        if (!b[i]) Ans = (LL)Ans * V % Mod, ++V;
```

```
        else if (b[i] != -1) V += 1 + (i == mxv);
```

```
    }
```

```
    printf("%d\n", Ans);
```

```
    return 0;
```

```
}
```

2 游戏

2.1 问题描述

维护一个序列 $a_1 \sim a_n$, 支持 q 次区间对 x 取 $\max$ 和区间询问。

具体的询问是, 给出区间和 x , 问 $a_l \sim a_r$ 再加上 x 作为石子堆, 玩 Nim 游戏, 可以让先手胜利的操作数量。

2.1.1 涉及的知识

博弈论, 数据结构。

2.1.2 数据规模与约定

表 2 数据规模与约定

测试点编号	$n, q \leq$	特殊性质
1	3000	无特殊性质
2		
3		
4		
5	2×10^5	无区间修改
6		所有修改在询问前执行
7		
8		
9		
10		修改的区间是 $[1, n]$
11		$a_i, x \leq 1$
12		
13		
14		
15		无特殊性质
16		
17		
18		
19		
20		

数据规模与约定如表 2 所示。对于所有的数据, $1 \leq n, q \leq 2 \times 10^5, 1 \leq l \leq r \leq n, 0 \leq a_i, x \leq 2^{30}$ 。

2.2 解题思路

Nim 游戏的性质众所周知, 设每个石子堆中石子个数为 $b_1, b_2, \dots, b_k$ 。令 $z = \bigoplus_{i=1}^k b_i$, 则先手取第 i 堆必胜当且仅当 $b_i \oplus z < b_i$ 。

考虑二进制, $b_i \oplus z < b_i$ 当且仅当 b_i 在 z 的最高位上为 1。对 $z = 0$ 可以特判。

回到原问题: 求出 $z = \bigoplus_{i=l}^r a_i$ 后, 令 j 为 z 的最

高位。问题转化为求区间内第 j 位为 1 的 a_i 的个数。

考虑修改操作, 区间对 x 取 $\max$ 是经典的 Segment Tree Beats (吉如一线段树, 见 IOI2016 中国国家候选队论文) 操作^[2]。

使用 Segment Tree Beats 维护序列, 需要支持询问区间异或和, 以及区间某一位为 1 的数的个数。

线段树每个结点维护最小值、最小值个数、次小值、区间异或和, 以及区间中第 j ($0 \leq j < 30$) 位为 1 的数的个数^[3]。

信息合并与标记下传的细节不再赘述。

时间复杂度为 $O(n \log n \log v)$ 。

2.3 总结

本题对数据结构中的经典算法进行了考察, 结合博弈论相关的基础知识, 综合考察了选手在代码实现中细节处理的多方面能力。

2.4 参考代码

```
#include <cstdio>
#include <climits>
#include <algorithm>

const int Inf = INT_MAX;
const int MN = 200005, MB = 30;

int N, Q, A[MN];

void Add(int *b, int x, int c) {
    for (int j = 0; j < MB; ++j) if (x >> j & 1)
        b[j] += c;
}

#define li (i << 1)
#define ri (li | 1)
#define mid ((l + r) >> 1)
#define ls li, l, mid
#define rs ri, mid + 1, r
const int MS = 1 << 19 | 7;
struct dat {
    int mn, cnt, mn2, val, num[MB];
    friend dat operator + (dat p, dat q) {
        if (q.mn < p.mn) {
```

```

        p.mn2 = std::min(p.mn, q.mn2);
        p.mn = q.mn;
        p.cnt = q.cnt;
    } else if (q.mn == p.mn) {
        p.cnt += q.cnt;
        p.mn2 = std::min(p.mn2, q.mn2);
    } else {
        p.mn2 = std::min(p.mn2, q.mn);
    }
    p.val ^= q.val;
    for (int i = 0; i < MB; ++i)
        p.num[i] += q.num[i];
    return p;
}
} tr[MS];
int tag[MS];
void P(int i, int x) {
    if (x <= tr[i].mn) return ;
    if (tr[i].cnt & 1) tr[i].val ^= tr[i].mn ^ x;
    Add(tr[i].num, tr[i].mn, -tr[i].cnt);
    Add(tr[i].num, tr[i].mn = x, tr[i].cnt);
    tag[i] = x;
}
void Pushdown(int i) { P(li, tag[i]), P(ri, tag[i]), tag[i]
= 0; }
void Build(int i, int l, int r) {
    if (l == r) {
        tr[i].val = tr[i].mn = A[l];
        tr[i].cnt = 1;
        tr[i].mn2 = Inf;
        Add(tr[i].num, A[l], 1);
        return ;
    }
    Build(ls), Build(rs);
    tr[i] = tr[li] + tr[ri];
}
void Mdf(int i, int l, int r, int a, int b, int x) {
    if (r < a || b < l) return ;
    if (a <= l && r <= b && x < tr[i].mn2) return P(i,
x);
    Pushdown(i);
    Mdf(ls, a, b, x), Mdf(rs, a, b, x);
    tr[i] = tr[li] + tr[ri];
}
int Qur1(int i, int l, int r, int a, int b) {
    if (r < a || b < l) return 0;
    if (a <= l && r <= b) return tr[i].val;
    Pushdown(i);
    return Qur1(ls, a, b) ^ Qur1(rs, a, b);
}
int Qur2(int i, int l, int r, int a, int b, int j) {
    if (r < a || b < l) return 0;
    if (a <= l && r <= b) return tr[i].num[j];
    Pushdown(i);
    return Qur2(ls, a, b, j) + Qur2(rs, a, b, j);
}
}
int main() {
    scanf("%d%d", &N, &Q);
    for (int i = 1; i <= N; ++i) scanf("%d", &A[i]);
    Build(1, 1, N);
    for (int q = 1; q <= Q; ++q) {
        int t, l, r, x;
        scanf("%d%d%d%d", &t, &l, &r, &x);
        if (t == 1) Mdf(1, 1, N, l, r, x);
        else {
            int z = x ^ Qur1(1, 1, N, l, r);
            if (!z) { puts("0"); continue; }
            int j = -1;
            for (int k = 0; k < MB; ++k) if (z >> k
& 1) j = k;
            printf("%d\n", Qur2(1, 1, N, l, r, j) +
(x >> j & 1));
        }
    }
    return 0;
}

```

3 滑冰

3.1 问题描述

一个 $n \times m$ 的地图，每个格子上可能有障碍物（默认边界外是障碍物）或标记点。

从给定的初始位置开始，选定四个方向之一，一直移动直到撞到障碍物停下，此时才可以继续选择方向移动。

问能否经过所有标记点。有多组数据

3.1.1 涉及的知识

性质挖掘，图论建模，2-SAT 问题。

3.1.2 数据规模与约定

数据规模与约定如表 3 所示。对于所有的数据，数据组数 ≤ 5 ， $1 \leq n, m \leq 50$ 。

表 3 数据规模与约定

测试点编号	特殊性质
1	地图中无障碍物
2	
3	
4	
5	$n = 1$ 或 $m = 1$
6	
7	标记点数不超过 1
8	
9	标记点数不超过 3
10	
11	标记点数不超过 5
12	
13	
14	无特殊性质
15	
16	
17	
18	
19	
20	
21	
22	
23	
24	
25	

3.2 解题思路

前两种数据（地图中无障碍物，或 $\min(n, m) = 1$ ）的规律很简单，大多数选手都能完成。

注意到对于一个方向，例如左方，可以用右左代替，在不改变位置的情况下经过更多格子（显然经过得越多越好）。

对于标记点 ≤ 5 的情况，可以枚举标记点经过的顺序（复杂度 $5!$ ）。

然后按经过顺序处理，可以枚举当前标记点是从左右方向经过还是从上下方向经过，然后 DFS 处理可达位置，查看是否能以左右方向或上下方向到达下一个标记点。

同时这档分也可以提示正解。

由前面的分析可知，标记点可分为左右方向经过或上下方向经过两种。因此可以利用左右或上下最长无障碍连续段作为结点。这样最多会有 $O(nm)$ 个结点。

结点之间可以形成有向转移图，如若上一次使用了左右方向的移动，则下一次可以在左端点或右端点处进行上下方向的移动。对于上下方向的情况也类似。

可以根据地图中的障碍物进行连边。再赋予一个起点，由于开始时可以选择左右还是上下方向，从这个起点向这两个结点连有向边。进行如上抽象后，相当于从起点出发，走一条可与自身相交的链，经过每个标记点。

如何定义“经过每个标记点”，一个标记点可以对应两个结点，即左右方向或上下方向，只要过程中经过其中任意一个即可。

这有点像 2-SAT 模型^[4]，即如果其中一个结点不选，则另一个结点必选。

如何进一步描述其他限制？如果两点之间互相都不可达，则它们之间不能都被选中。只需要满足此限制就一定存在路径吗？换句话说，就是满足两点 a, b 之间一定要么 $a \rightarrow b$ 要么 $b \rightarrow a$ 要么 $a \leftrightarrow b$ 。

也就是说，这是一个全序关系，将 $a \leftrightarrow b$ 看成强连通分量（等价类）。由于全序关系的性质，一定能找到一条经过所有结点的链。因此上述两个限制就是需要的所有限制。

求任意两点可达性可以简单进行 $O((nm)^2)$ 的 DFS。最后建出的 2-SAT 图的边数也是这个级别的，求解 2-SAT 即可。

总时间复杂度为 $O((nm)^2)$ 。

3.3 总结

本题解法巧妙,部分分也有对正解给予提示,考察选手的思维能力和图论知识。

本场比赛三题难度并不高,分别考察选手的组计数、数据结构和图论,难度曲线平缓,区分度良好,同时注重选手的思维能力。

3.4 参考代码

```
#include <cstdio>
#include <algorithm>
#include <vector>
#include <stack>

const int MN = 55, MC = 2505;

int N, M, C;
char A[MN][MN];
int Rid[MN][MN], Cid[MN][MN];

bool TC[MC][MC];
std::vector<int> G[MC];
inline void AddEdge(int x, int y) {
    G[x].push_back(y);
}
void DFS(int u, int s) {
    TC[s][u] = true;
    for (int v : G[u])
        if (!TC[s][v]) DFS(v, s);
}

struct TwoSAT {
    int V;
    std::vector<std::vector<int>>> G;
    void init(int n) {
        V = n;
        G.clear(), G.resize(2 * V + 1);
    }
    TwoSAT() { init(0); }
    TwoSAT(int n) { init(n); }
    void _addEdge(int u, int v) {
```

```
        if (u < 0) u = V - u;
        if (v < 0) v = V - v;
        G[u].push_back(v);
    }
    void addEdge(int u, int v) {
        _addEdge(u, v);
        if (u != -v) _addEdge(-v, -u);
    }

    int dfc, sent;
    std::stack<int> stk;
    std::vector<int> dfn, low, bel;
    std::vector<bool> instk;
    void Tarjan(int u) {
        dfn[u] = ++dfc;
        low[u] = dfn[u];
        stk.push(u), instk[u] = true;
        for (int v : G[u]) {
            if (!dfn[v]) {
                Tarjan(v);
                low[u] = std::min(low[u], low[v]);
            } else if (instk[v])
                low[u] = std::min(low[u], dfn[v]);
        }
        if (low[u] == dfn[u]) {
            ++sent;
            while (true) {
                int x = stk.top();
                bel[x] = sent;
                instk[x] = false;
                stk.pop();
                if (x == u) break;
            }
        }
    }
    bool Solve() {
        dfc = 0, sent = 0;
        dfn.resize(2 * V + 1);
```

```

        low.resize(2 * V + 1);
        bel.resize(2 * V + 1);
        instk.resize(2 * V + 1);
        for (int i = 1; i <= 2 * V; ++i) if (!dfn[i])
Tarjan(i);
        for (int i = 1; i <= V; ++i) if (bel[i] == bel[V
+ i]) return false;
        return true;
    }
};

int main() {
    int Tests;
    scanf("%d", &Tests);
    while (Tests--) {
        scanf("%d%d", &N, &M);
        for (int i = 1; i <= N; ++i)
            scanf("%s", A[i] + 1);
        for (int i = 1; i <= N; ++i) A[i][0] = A[i][M
+ 1] = '#';
        for (int j = 1; j <= M; ++j) A[0][j] = A[N +
1][j] = '#';
        C = 0;
        for (int i = 1; i <= N; ++i)
            for (int j = 1; j <= M; ++j) if (A[i][j] !=
'#') {
                Rid[i][j] = A[i][j - 1] == '#' ?
++C : Rid[i][j - 1];
                Cid[i][j] = A[i - 1][j] == '#' ?
++C : Cid[i - 1][j];
            }
        for (int i = 0; i <= C; ++i) G[i].clear();
        for (int i = 1; i <= N; ++i)
            for (int j = 1; j <= M; ++j) if (A[i][j] !=
'#') {
                if (A[i][j - 1] == '#' || A[i][j + 1]
== '#') AddEdge(Rid[i][j], Cid[i][j]);
                if (A[i - 1][j] == '#' || A[i + 1][j]

```

```

== '#') AddEdge(Cid[i][j], Rid[i][j]);
                if (A[i][j] == 'S') AddEdge(0,
Rid[i][j]), AddEdge(0, Cid[i][j]);
            }
        for (int i = 0; i <= C; ++i)
            for (int j = 0; j <= C; ++j)
                TC[i][j] = false;
        for (int i = 0; i <= C; ++i) DFS(i, i);
        TwoSAT x(C);
        for (int i = 1; i <= C; ++i)
            for (int j = 1; j <= C; ++j) if (!TC[i][j]
&& !TC[j][i])
                x.addEdge(i, -j);
        for (int i = 1; i <= C; ++i) if (!TC[0][i])
            x.addEdge(i, -i);
        for (int i = 1; i <= N; ++i)
            for (int j = 1; j <= M; ++j) if (A[i][j] ==
'o')
                x.addEdge(-Rid[i][j], Cid[i][j]);
        if (x.Solve()) puts("Yes");
        else puts("No");
    }
    return 0;
}

```

参 考 文 献

- [1] 刘汝佳.算法竞赛入门经典.第2版.北京:清华大学出版社,2014
- [2] 吉如一.区间最值操作与历史最值问题. 2016 年信息学奥林匹克中国国家队候选队员论文集,2016
- [3] 王晓东.数据结构: C语言描述.第3版.北京:电子工业出版社,2019
- [4] 王晓东.计算机算法设计与分析.第5版.北京:电子工业出版社,2018